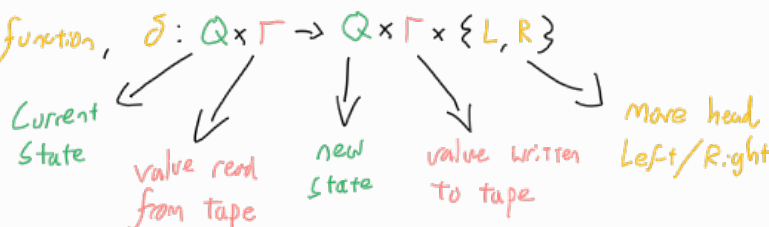


Turing Machines

Turing Machines are a mathematically rigorous definition of Computation that can do anything a real computer can do (Church-Turing Thesis).

A TM has an **infinite tape** (memory) and a **read/write head** that can move in both directions on the tape, controlled by a **finite state program**. This program can be thought of as a FSM. State transitions are determined by the current state and the contents read by the **head** in its current position, and these transitions control what the head writes and whether it moves to the left or right.

Formally, a TM is a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ such that:

- Q is the set of **program states**
- Σ is the **input alphabet** - Not containing $\sqcup$
- Γ is the **tape alphabet** $= \Sigma \cup \sqcup$
- δ is the **transition function**, $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$


- q_0 is the **start state**, $q_0 \in Q$

- q_{accept} and q_{reject} are the **accept** and **reject states** respectively, $q_{\text{accept}}, q_{\text{reject}} \in Q$, $q_{\text{accept}} \neq q_{\text{reject}}$

The tape content is unbounded but always finite.

A Configuration consists of the **current state**, the tape content and the **head position**.

A Configuration C_1 yields C_2 if $C_1 \rightarrow C_2$ in a single step.

When running a TM on some input $w \in \Sigma^*$ the start configuration consists of the start state q_0 , the tape content being w (starting at the leftmost position on the tape), and the **head** being at the leftmost position.

Note: if the **head** is at the leftmost position and a state transition makes it move **Left**, it stays where it is.

Accepting and **Rejecting** configurations are **halting** conditions whose state is $q_{\text{accept}} / q_{\text{reject}}$.

A TM **accepts** an input w if there exists some sequence of configurations $C_1 \rightarrow C_2 \rightarrow \dots \rightarrow C_{\text{accept}}$ where every configuration yields the next.

The set of strings accepted by a TM, M , is known as the **language of M** , denoted $L(M)$.

A language L is **Turing-Recognisable** or **Semi-Decidable** if there exists a TM M that recognises it, i.e. $L = L(M)$. This means that M accepts and always halts for all words in L , and either rejects or fails to halt on words not in L .

A language L is **Turing-Decidable** (or **Decidable**) if there exists a TM M that accepts all words in L and rejects all words not in L - always halts.

Other Types of TM:

Multitape TM - one with multiple heads, each with its own head. The only difference from a normal TM is the **Transition function**, which for a k -tape TM is:

$$\delta: Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R\}^k$$

Every multitape TM is equivalent to some single-tape TM.

Non-deterministic TM - a TM that accepts if some sequence of configurations exists that leads to an accepting configuration. The **Transition function** is

$$\delta: Q \times \Gamma \rightarrow P(Q \times \Gamma \times \{L, R\})$$



Power Set = Set of all possibilities.

Every Non-deterministic TM has an equivalent deterministic (normal) TM.
 $\Rightarrow$ perform breadth-first search on the tree of all possible computations and accept if some accepting configuration is found.

Universal TM - Every TM M can be encoded as a word on a finite alphabet - this encoding is denoted $\langle M \rangle$. A Universal TM is a TM that takes $\langle M \rangle$ and some word w as input, and simulates M on w .

Halting Problem - given the encoding $\langle M \rangle$ of a Turing machine M , and a word w , does M terminate on w ?

The Halting Problem is **Turing-Recognisable**.

$\rightarrow$ run a universal TM on $(\langle M \rangle, w)$ and accept if this computation terminates.

The Halting Problem is not **Turing-Decidable**

Proof. Assume **for contradiction** that there is a TM $\mathcal{H}$ that decides the Halting Problem

$$\mathcal{H}(\langle M \rangle, w) = \begin{cases} \text{accept} & \text{if } M \text{ terminates on } w \\ \text{reject} & \text{if } M \text{ does not terminate on } w. \end{cases}$$

I will use as a black-box it to create an instance of the Halting Problem, on which $\mathcal{H}$ fails.

Consider a TM $\mathcal{D}$ that takes (the description of) a single TM M as an input and does the following:

$$\mathcal{D}(\langle M \rangle) = \begin{cases} \text{accept} & \text{if } \mathcal{H}(\langle M \rangle, \langle M \rangle) \text{ rejects} \\ \text{loop} & \text{if } \mathcal{H}(\langle M \rangle, \langle M \rangle) \text{ accepts.} \end{cases}$$

We think of $\mathcal{D}$ being the following program:

L : if $\mathcal{H}(\langle \cdot \rangle, \langle \cdot \rangle)$ then go to L else return *true*.

What happens when $\mathcal{D}$ runs on its own encoding $\langle \mathcal{D} \rangle$? There are two possibilities:

1. $\mathcal{D}$ terminates on $\langle \mathcal{D} \rangle$. By the construction of $\mathcal{D}$, we have that $\mathcal{H}(\langle \mathcal{D} \rangle, \langle \mathcal{D} \rangle)$ rejects, thus gives a wrong answer.
2. $\mathcal{D}$ does not terminate on $\langle \mathcal{D} \rangle$. By the construction of $\mathcal{D}$, we have that $\mathcal{H}(\langle \mathcal{D} \rangle, \langle \mathcal{D} \rangle)$ accepts, so $\mathcal{H}$ gives a wrong answer in this case, too.

The complement, $\text{Co-}L$, of a language L , is the language that accepts all words over the same alphabet that L does not accept.
 $\Rightarrow L$ is decidable iff both L and $\text{Co-}L$ are recognisable.

The Co-Halting Problem is not recognisable, since it does not halt on any accepted words.

The Step-Counter Predicate is recognisable as it accepts all words such that some TM M terminates on them in $\leq k$ steps for some finite k .

The Co-Step-Counter problem is also recognisable since we can terminate after $k+1$ steps for all accepting inputs.

$\Rightarrow$ Step-Counter Predicate is decidable.